Codex Working Brief — robotics-maze-g1

You are working inside my repository:

```
~/dev_workspace/robotics-maze-g1
```

This path may be a symlink on my laptop to:

```
/mnt/robotics_ssd/dev_workspace/robotics-maze-g1
```

That is only my local machine setup. Do **not** hardcode `/mnt/robotics_ssd`, `/media/robojoe`, `/home/robojoe`, or any machine-specific path into the repo.

Project goal

This is a robotics simulation project for a Unitree G1 / humanoid maze navigation assignment.

The repo should be easy to run reproducibly on different machines using Docker.

The Docker environment should:

- Use Ubuntu 22.04.
- Use ROS 2 Humble.
- Include Nav2 and SLAM/navigation-related packages.
- Not require ROS installed on the host.
- Support x86 laptops and later ARM/Jetson machines through multi-arch Docker support.
- Preserve the existing `make` workflow.
- Support visible simulation/debugging when display support exists.
- Keep non-GUI/headless testing available.

Important local-machine context

My laptop has a tiny Ubuntu root partition, so Docker storage has been moved locally to an external SSD.

On my laptop only, Docker may use:

```
/mnt/robotics_ssd/docker-data
```

This is configured outside the repo in:

```
/etc/docker/daemon.json
```

Do not modify `/etc/docker/daemon.json`.

Do not modify `/etc/fstab`.

Do not modify mount points.

Do not run destructive system commands.

Do not assume other users have `/mnt/robotics_ssd`.

The project must remain portable. Other machines should be able to clone the repo and run normal Docker commands with their local Docker storage.

Safety rules

Before making changes:

1. Inspect current status:

```
git status --short
git branch --show-current
```

1. Search for machine-specific paths:

```
grep -R "/mnt/robotics_ssd\|/media/robojoe\|/home/robojoe" . --exclude-
dir=.git || true
```

If these appear in repo files, replace them with repo-relative paths or environment variables.

1. Do not edit files outside the repository.
2. Do not delete large folders outside the repository.
3. Do not install host-level packages.
4. Do not change Docker daemon/system configuration.
5. Preserve all existing `make` targets.
6. Do not remove existing functionality unless it is clearly broken and you explain why.

Current desired workflow

The normal project commands should remain:

```
make docker-build
make docker-run
make docker-run-gui
make docker-test
make docker-smoke
make docker-check-ros
```

The repo should also continue supporting existing non-Docker commands where applicable.

First task

Validate the Docker workflow from the repository level.

Run:

```
make docker-build
```

If it fails, inspect the Dockerfile, Makefile, and scripts under `docker/`.

Fix repo-level issues only.

Then run:

```
make docker-check-ros
make docker-smoke
make docker-test
```

Only after those pass, test:

```
make docker-run
```

Then test GUI last:

```
make docker-run-gui
```

If GUI fails, diagnose it separately as an X11/display issue. Do not rewrite the project just because GUI fails.

Expected Docker design

The repo should include or preserve:

```
docker/Dockerfile
docker/entrypoint.sh
docker/run.sh
docker/run_gui.sh
docker/build_multiarch.sh
.dockerignore
```

The Docker image should be based on:

```
FROM osrf/ros:humble-desktop
```

unless there is a strong technical reason not to.

It should include ROS/navigation packages such as:

```
ros-humble-navigation2
ros-humble-nav2-bringup
ros-humble-slam-toolbox
ros-humble-depthimage-to-laserscan
ros-humble-robot-localization
ros-humble-tf-transformations
ros-humble-image-transport
ros-humble-cv-bridge
ros-humble-vision-msgs
ros-humble-rtabmap-ros
rviz2
rqt
rqt-common-plugins
```

Also include normal build/debug dependencies needed by the project:

```
build-essential
cmake
git
wget
curl
nano
vim
tmux
bash-completion
python3-pip
python3-venv
python3-colcon-common-extensions
python3-rosdep
python3-vcstool
ffmpeg
mesa-utils
```

```
libgl1
libgl1-mesa-dri
libglfw3
libglfw3-dev
libglew-dev
libosmesa6
libosmesa6-dev
```

Install Python dependencies from `requirements.txt` if present.

Run script expectations

`docker/run.sh` should:

- Mount the current repo into the container.
- Use the repo root dynamically, not a hardcoded path.
- Support optional command arguments, e.g.:

```
docker/run.sh make test
```

- Use host networking if needed for ROS 2.
- Pass `ROS_DOMAIN_ID`.
- Work without GUI.

`docker/run_gui.sh` should:

- Support X11 display forwarding.
- Mount `/tmp/.X11-unix`.
- Pass `DISPLAY`.
- Set `QT_X11_NO_MITSHM=1`.
- Avoid hardcoded user paths.
- Fail with a clear message if display support is unavailable.

Makefile expectations

The Makefile should preserve existing targets and include Docker targets like:

```
docker-build
docker-run
docker-run-gui
docker-test
docker-smoke
docker-check-ros
docker-build-multiarch
```

Default variables may include:

```
DOCKER_IMAGE ?= robotics-maze-g1:humble
ROS_DOMAIN_ID ?= 0
DOCKER_PLATFORMS ?= linux/amd64,linux/arm64
```

Validation script expectation

If `scripts/check_ros_docker_env.sh` exists, make sure it checks useful things inside the container:

```
echo $ROS_DISTRO
python3 --version
ros2 --help
ros2 pkg list | grep nav2
ros2 pkg list | grep slam_toolbox
ros2 pkg list | grep depthimage_to_laserscan
```

It should fail clearly if ROS packages are missing.

Project behavior expectations

Do not rewrite the robotics/navigation architecture unless needed for Docker compatibility.

Do not change maze generation, navigation, locomotion, milestone logic, or KPI logic unless the test failure directly requires it.

I always need visible results where possible, but there should also be a headless mode for testing.

After each change

Run the smallest relevant test first.

Then run:

```
make docker-check-ros
make docker-smoke
make docker-test
```

Before finishing, provide:

1. What changed.
2. Why it changed.
3. Commands run.
4. What passed.
5. What failed, if anything.
6. Exact next steps for me.

Do not do these things

Do not modify:

```
/etc/fstab
/etc/docker/daemon.json
/mnt/robotics_ssd
/media/robojoe
/var/lib/docker
/var/lib/containerd
```

Do not run:

```
sudo rm -rf
sudo apt install
sudo mount
sudo systemctl
```

unless I explicitly ask outside Codex.

This Codex session should focus on repository work only.